MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Done by: st. gr. FAF-242, Temciuc Adelina

Report

on laboratory work no. 6
Operating Systems
Topic: Inside the xv6 File System

Checked by:
Reiţman Patricia, university assistant
DISA, FCIM, UTM

Chișinău – 2026

AIM OF THE WORK

Purpose of the laboratory work

The purpose of this laboratory work is to see the ideas of a Unix file system (inodes, directories as tables of names, hard links and fixed limits) in the small and readable file system of xv6. The information that xv6 prints is decoded with the help of the kernel headers, two new user programs are written, `mystat` and `find`, and the limits written in `kernel/fs.h` are measured and calculated.

The laboratory work aims to:

- decode the output of the xv6 `ls` (type, inode number, size) using `kernel/stat.h`;
- write the program `mystat`, which prints the type, inode, link count and size of files, and use it to watch hard links at work;
- find in the kernel where the link count is decreased and when the data blocks are freed, and list the fields of an on-disk inode;
- measure the limits of the file system: the length of a name, the largest file and the layout of the disk;
- write the program `find`, which walks the directory tree recursively.

THEORETICAL BACKGROUND

The xv6 file system

The xv6 file system is built in layers: the virtio disk driver, a buffer cache, a log (journal) that makes multi-block updates atomic, inodes, directories and path names, and finally file descriptors. The disk is divided into blocks of `BSIZE` = 1024 bytes: block 0 is the boot block, block 1 the superblock, then come the log, the inode blocks, the free-block bitmap and finally the data blocks. The image `fs.img` is created on the host by the program `mkfs` during `make`.

Inodes and directories

An on-disk inode (`struct dinode`) stores the type, the device numbers, the link count, the size and the addresses of the data blocks: 12 direct block numbers and one indirect block that contains more block numbers. A directory is an ordinary file whose content is an array of `struct dirent` entries, each holding an inode number and a name of at most `DIRSIZ` characters. An entry with inode number 0 is free.

Links and stat

A hard link is one more directory entry pointing to the same inode; the link count `nlink` in the inode counts these names. `unlink` removes a name and decreases the count; the inode and its blocks are freed only when the count is 0 and no process uses the file any more. The system call `fstat` and the library function `stat()` copy the main fields of an inode into a `struct stat` (`dev`, `ino`, `type`, `nlink`, `size`), which is how user programs such as `ls` learn about files. Unlike Linux, xv6 has no owners, permissions, timestamps or symbolic links.

PART 1. READ WHAT XV6 SHOWS YOU

1.1 Commands executed

xv6 was started with `make qemu` in `~/xv6-riscv`, and the following commands were run at the `$` prompt:

```
ls
mkdir a
ls a
```

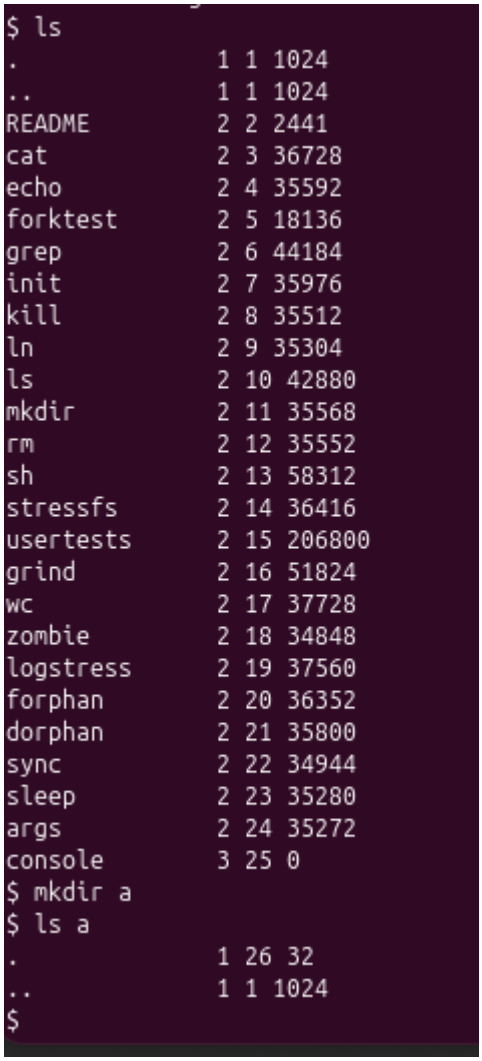


Figure 1.1 – ls in the root directory and in the new directory a

Each line of `ls` shows the name, the type, the inode number and the size in bytes. At this point the tree still contained the programs `sleep` and `args` from Lab 4, so `console` had inode 25. The types are defined in `kernel/stat.h`:

```
#define T_DIR    1   // Directory
#define T_FILE   2   // File
#define T_DEVICE 3   // Device
```

1.2 Answers to the questions

1. What do types 1, 2 and 3 mean?

Type 1 is `T_DIR`, a directory (`.`, `..` and `a`); type 2 is `T_FILE`, a regular file (`README` and all programs); type 3 is `T_DEVICE`, a device (`console`, whose size is 0 because it has no data blocks, only device numbers).

2. Which inode does the root directory have, and why are . and .. the same there?

The root directory has inode 1 (`ROOTINO` in `kernel/fs.h`). Both `.` and `..` show `1 1 1024`, because the root has no parent: `mkfs` writes the entry `..` of the root so that it points to the root itself, and the kernel cannot go higher than `/`. Inode 0 is never used, because the number 0 marks a free directory entry.

3. How big is one directory entry, and which two entries does a contain?

A directory entry is `struct dirent` = `ushort inum` (2 bytes) + `char name[DIRSIZ]` (14 bytes) = 16 bytes. The empty directory `a` has size 32 = 2 × 16, so it contains exactly two entries: `.` with inode 26 (the directory `a` itself) and `..` with inode 1 (the root, its parent), as `ls a` shows.

PART 2. WRITE MYSTAT

2.1 The program user/mystat.c

The function `typename()` turns the type into a word with a `switch` on `T_DIR`, `T_FILE` and `T_DEVICE`. `main()` calls `stat()` for every argument; if it fails, an error is printed to file descriptor 2 and the loop continues with the next argument. Otherwise one line is printed in the format `NAME: TYPE, inode N, links N, size N bytes`. Without arguments the program prints a usage message.

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

char *
typename(int t)
{
  switch(t){
  case T_DIR:
    return "directory";
  case T_FILE:
    return "file";
  case T_DEVICE:
    return "device";
  }
  return "?";
}

int
main(int argc, char *argv[])
{
  struct stat st;

  if(argc < 2){
    fprintf(2, "usage: mystat path...\n");
    exit(1);
  }
  for(int i = 1; i < argc; i++){
    if(stat(argv[i], &st) < 0){
      fprintf(2, "mystat: cannot stat %s\n", argv[i]);
      continue;
    }
    printf("%s: %s, inode %d, links %d, size %d bytes\n",
           argv[i], typename(st.type), st.ino, st.nlink, (int)st.size);
  }
  exit(0);
}
```

The lines `$U/_mystat\` and `$U/_find\` (for Part 4) were added to `UPROGS` in the Makefile after the programs from Lab 4, each starting with a TAB and with nothing after the backslash.

```
vboxuser@Labubu:~/xv6-riscv$ cat user/mystat.c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

char *
typename(int t)
{
  switch(t){
  case T_DIR:
    return "directory";
  case T_FILE:
    return "file";
  case T_DEVICE:
    return "device";
  }
  return "?";
}

int
main(int argc, char *argv[])
{
  struct stat st;

  if(argc < 2){
    fprintf(2, "usage: mystat path...\n");
    exit(1);
  }
  for(int i = 1; i < argc; i++){
    if(stat(argv[i], &st) < 0){
      fprintf(2, "mystat: cannot stat %s\n", argv[i]);
      continue;
    }
    printf("%s: %s, inode %d, links %d, size %d bytes\n",
           argv[i], typename(st.type), st.ino, st.nlink, (int)st.size);
  }
  exit(0);
}
vboxuser@Labubu:~/xv6-riscv$
```

Figure 2.1 – The source code of mystat.c

```
vboxuser@Labubu:~/xv6-riscv$ grep -n "_sync\|_sleep\|_args\|_mystat\|_find" Makefile
152:        $U/_sync\
153:        $U/_sleep\
154:        $U/_args\
155:        $U/_mystat\
156:        $U/_find\
vboxuser@Labubu:~/xv6-riscv$
```

Figure 2.2 – The new lines in UPROGS

2.2 Watching hard links with mystat

After `make qemu` (the disk was rebuilt, so `console` now has inode 27 after `mystat` and `find`), the following commands were run:

```
mystat README . console
ln README r2
mystat README r2
rm README
mystat r2
cat r2 | wc
mystat nosuchfile a
```

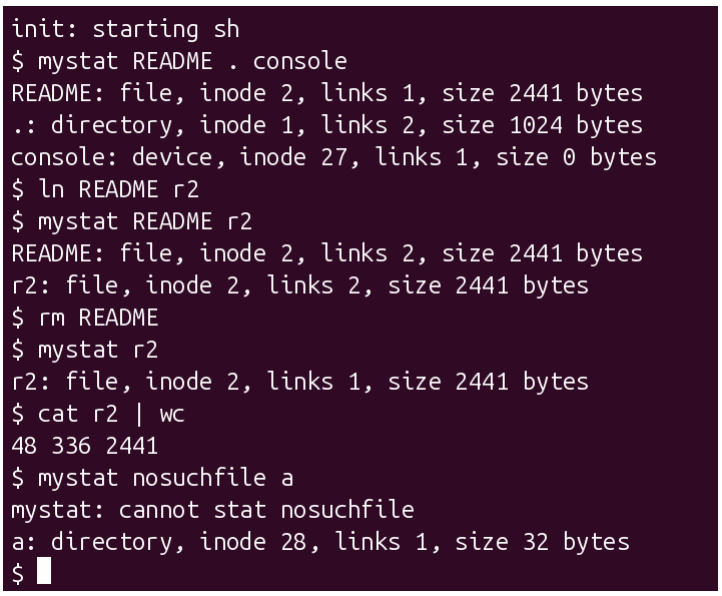


Figure 2.3 – mystat before and after ln and rm

2.3 Answers to the questions

1. The outputs after ln and after rm. What changed and what did not?

After `ln README r2`:

```
README: file, inode 2, links 2, size 2441 bytes
r2: file, inode 2, links 2, size 2441 bytes
```

After `rm README`:

```
r2: file, inode 2, links 1, size 2441 bytes
```

`ln` did not create a new file: both names show the same inode 2 and the same size, only the link count went from 1 to 2. After `rm README` the name `README` disappeared and the count went back to 1, but the inode number, the type, the size and the data stayed the same: `cat r2 | wc` still counted 48 lines, 336 words and 2441 bytes. Only the link count changes; everything else belongs to the inode, not to the name.

In this version of xv6 `mystat .` showed `links 2` for the root (the task sheet shows 1): the current kernel increases the link count of a parent directory for the `..` of each new subdirectory (`dp->nlink++; // for ".."` in `create()`), and the directory `a` had been created. The test `mystat`

`nosuchfile a` also shows the error path: the message goes to standard error and the program continues with the next argument.

2. The line in sys_unlink that decrements the link count. When are the data blocks freed?

```
vboxuser@Labubu:~/xv6-riscv$ grep -n "nlink--" kernel/sysfile.c
178:  ip->nlink--;
239:    dp->nlink--;
244:  ip->nlink--;
vboxuser@Labubu:~/xv6-riscv$ grep -n -A 7 "^struct dinode {" kernel/fs.h
32:struct dinode {
33-  short type;               // File type
34-  short major;              // Major device number (T_DEVICE only)
35-  short minor;              // Minor device number (T_DEVICE only)
36-  short nlink;              // Number of links to inode in file system
37-  uint size;                // Size of file (bytes)
38-  uint addrs[NDIRECT + 1]; // Data block addresses
39-};
vboxuser@Labubu:~/xv6-riscv$
```

Figure 2.4 – nlink-- in sysfile.c and struct dinode in fs.h

In `kernel/sysfile.c`, `sys_unlink()` decrements the link count of the file on line 244: `ip->nlink--;` followed by `iupdate(ip);`, which writes the inode to disk. Line 239 (`dp->nlink--`) decreases the count of the parent when a directory is removed, and line 178 is the error path of `sys_link()`.

The blocks are not freed in `sys_unlink()`. They are freed in `iput()` in `kernel/fs.c`, when the last in-memory reference to the inode is released and its link count is 0: then `itrunc()` frees all data blocks (and the indirect block) and the inode is marked free. So if a process still has the file open, the data survives the `rm` and is freed only when that process closes the file, just like on Linux.

3. Which fields does an xv6 inode have?

`struct dinode` in `kernel/fs.h` has only six fields: `type` (file, directory, device or 0 for free), `major` and `minor` (device numbers, only for `T_DEVICE`), `nlink` (number of names), `size` (in bytes) and `addrs[NDIRECT + 1]` (12 direct block addresses and one indirect block). Compared with ext4 there is no owner, no group, no permission bits, no timestamps and no symbolic-link type, which is why xv6 has no permissions and no symbolic links.

PART 3. MEASURE THE LIMITS

3.1 Commands executed

On the host, in `~/xv6-riscv`:

```
grep -n "define BSIZE\|define NDIRECT\|define NINDIRECT\|define DIRSIZ"
kernel/fs.h
make clean ; make fs.img 2>&1 | grep nmeta
```

```
vboxuser@Labubu:~/xv6-riscv$ grep -n "define BSIZE\|define NDIRECT\|define NINDIRECT\|define DIRSIZ" kernel/fs.h
5:#define BSIZE   1024 // block size
26:#define NDIRECT   12
27:#define NINDIRECT (BSIZE / sizeof(uint))
54:#define DIRSIZ 14
vboxuser@Labubu:~/xv6-riscv$ make clean ; make fs.img 2>&1 | grep nmeta
rm -f *.tex *.dvi *.idx *.aux *.log *.ind *.ilg \
*/*.o */*.d */*.asm */*.sym \
kernel/kernel fs.img \
mkfs/mkfs .gdbinit \
        user/usys.S \
user/_cat user/_echo user/_forktest user/_grep user/_init user/_kill user/_ln user/_ls user/_mkdir
user/_rm user/_sh user/_stressfs user/_usertests user/_grind user/_wc user/_zombie user/_logstress
user/_forphan user/_dorphan user/_sync user/_sleep user/_args user/_mystat user/_find
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000
vboxuser@Labubu:~/xv6-riscv$
```

Figure 3.1 – The constants in fs.h and the disk layout printed by mkfs

Then on a fresh disk in xv6:

```
mkdir averyveryverylongname
ls
mystat averyveryverylongname averyveryveryl
```

```
init: starting sh
$ mkdir averyveryverylongname
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 36728
echo           2 4 35592
forktest       2 5 18136
grep           2 6 44184
init           2 7 35976
kill           2 8 35512
ln             2 9 35304
ls             2 10 42880
mkdir          2 11 35568
rm             2 12 35552
sh             2 13 58312
stressfs       2 14 36416
usertests      2 15 206800
grind          2 16 51824
wc             2 17 37728
zombie         2 18 34848
logstress      2 19 37560
forphan        2 20 36352
dorphan        2 21 35800
sync           2 22 34944
sleep          2 23 35280
args           2 24 35272
mystat         2 25 36296
find           2 26 42952
console        3 27 0
averyveryveryl 1 28 32
$ mystat averyveryverylongname averyveryveryl
averyveryverylongname: directory, inode 28, links 1, size 32 bytes
averyveryveryl: directory, inode 28, links 1, size 32 bytes
```

Figure 3.2 – A directory name longer than DIRSIZ

3.2 Answers to the questions

1. What does ls show for averyveryverylongname, and which constant explains it?

`ls` shows only `averyveryveryl`, the first 14 characters (type 1, inode 28, size 32). The constant is `DIRSIZ` = 14: the name field of a directory entry has room for 14 characters, so `mkdir` silently cut the name. When names are looked up, the kernel compares at most `DIRSIZ` characters (`namecmp()` uses `strncmp(s, t, DIRSIZ)`), so both the long and the short name lead to the same inode 28, as `mystat` shows.

2. The largest file xv6 can hold

An inode has `NDIRECT` = 12 direct blocks. The indirect block is one block of `BSIZE` = 1024 bytes filled with block numbers of 4 bytes, so it holds `NINDIRECT` = 1024 / 4 = 256 more block numbers.

```
MAXFILE = NDIRECT + NINDIRECT = 12 + 1024 / 4 = 12 + 256 = 268 blocks
```

```
max size = 268 × BSIZE = 268 × 1024 = 274 432 bytes = 268 KB
```

So the largest file in xv6 is 274 432 bytes, about a quarter of a megabyte. Larger files would need a double-indirect block, as in ext2 or in the "large files" exercise of the MIT course.

3. The disk layout from the nmeta line

The line printed by `mkfs` was `nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000`.

Area	Blocks	Explanation
boot block	1	block 0, not used by xv6
superblock	1	block 1, describes the layout of the disk
log	31	1 header block + `LOGBLOCKS` = 30 data blocks
inodes	13	200 inodes, 16 inodes of 64 bytes per block: 200 / 16 + 1 = 13
bitmap	1	one block of 8192 bits is enough for 2000 blocks
metadata total (nmeta)	47	1 + 1 + 31 + 13 + 1
data	1953	2000 − 47, for the contents of files and directories
whole disk	2000	`FSSIZE` = 2000 blocks of 1 KB = 2 000 KB (`fs.img`)

PART 4. WRITE FIND

4.1 The program user/find.c

The program is based on `user/ls.c`. The function `find(path, name)` opens the directory, reads `struct dirent` entries one by one, skips free entries (`inum == 0`) as well as `.` and `..`, builds the full path in a buffer and calls `stat()` on it. If the last component equals the searched name (compared with `strcmp`), the path is printed; if the entry is a directory, `find()` calls itself on it. Every directory is closed before the function returns, so the program never has more open files than the depth of the tree, well under the limit of 16. The optional third argument `-n` (the stretch task) also prints the inode number of each match.

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
#include "kernel/fs.h"
#include "kernel/fcntl.h"

int shownum;   // set by the optional -n argument

// Return the part of path after the last slash.
char *
lastname(char *path)
{
  char *p;

  for(p = path + strlen(path); p >= path && *p != '/'; p--)
    ;
  return p + 1;
}

void
find(char *path, char *name)
{
  char buf[512], *p;
  int fd;
  struct dirent de;
  struct stat st;

  if((fd = open(path, O_RDONLY)) < 0){
    fprintf(2, "find: cannot open %s\n", path);
    return;
  }
  if(fstat(fd, &st) < 0){
    fprintf(2, "find: cannot stat %s\n", path);
    close(fd);
```

```
    return;
  }
  if(st.type != T_DIR){
    close(fd);
    return;
  }
  if(strlen(path) + 1 + DIRSIZ + 1 > sizeof buf){
    fprintf(2, "find: path too long\n");
    close(fd);
    return;
  }

  strcpy(buf, path);
  p = buf + strlen(buf);
  *p++ = '/';
  while(read(fd, &de, sizeof(de)) == sizeof(de)){
    if(de.inum == 0)
      continue;
    memmove(p, de.name, DIRSIZ);
    p[DIRSIZ] = 0;
    if(strcmp(p, ".") == 0 || strcmp(p, "..") == 0)
      continue;
    if(stat(buf, &st) < 0){
      fprintf(2, "find: cannot stat %s\n", buf);
      continue;
    }
    if(strcmp(p, name) == 0){
      if(shownum)
        printf("%s %d\n", buf, st.ino);
      else
        printf("%s\n", buf);
    }
    if(st.type == T_DIR)
      find(buf, name);
  }
  close(fd);
}

int
main(int argc, char *argv[])
{
  if(argc < 3 || argc > 4 || (argc == 4 && strcmp(argv[3], "-n") != 0)){
    fprintf(2, "usage: find <dir> <name> [-n]\n");
    exit(1);
  }
  shownum = (argc == 4);
  find(argv[1], argv[2]);
  exit(0);
}
```

4.2 Testing on a fresh disk

The tests from the task were run after `make clean ; make qemu` (continuing the session of Figure 3.2), followed by the stretch test and a check with `mystat`:

```
mkdir a
mkdir a/b
echo x > a/b/r2
echo y > a/r2
ln cat r2
find . r2
find a b
find . r2 -n
mystat cat r2
find
```

```
$ mkdir a
$ mkdir a/b
$ echo x > a/b/r2
$ echo y > a/r2
$ ln cat r2
$ find . r2
./a/b/r2
./a/r2
./r2
$ find a b
a/b
$ find . r2 -n
./a/b/r2 31
./a/r2 32
./r2 3
$ mystat cat r2
cat: file, inode 3, links 2, size 36728 bytes
r2: file, inode 3, links 2, size 36728 bytes
$ find
usage: find <dir> <name> [-n]
$
```

Figure 4.1 – find on the test tree, with and without -n

`find . r2` printed the three expected paths `./a/b/r2`, `./a/r2` and `./r2`, and `find a b` printed `a/b`, which shows that the program goes down into subdirectories and does not loop through `.` or `..`. Without arguments it prints the usage message.

4.3 Stretch: which r2 is the same file as cat?

With `-n`, the three matches have the inode numbers 31, 32 and 3. `./r2` has inode 3, the same as `cat`, because it was created with `ln cat r2`: `mystat cat r2` confirms that both names point to inode 3 with 2 links and the size of the `cat` program. `./a/b/r2` (inode 31) and `./a/r2` (inode 32) are separate small files created by `echo`.

CONCLUSION

In this laboratory work the file system of xv6 was studied from the user side and from the kernel source.

In the first part, the output of `ls` was decoded with `kernel/stat.h`: types 1, 2 and 3 are directory, file and device, the root directory has inode 1 and is its own parent, and a directory entry has 16 bytes, so an empty directory with `.` and `..` is 32 bytes long.

In the second part, the program `mystat` was written and used to show that a hard link adds only a name: after `ln` both names had inode 2 and 2 links, and after `rm` the data stayed and only the count went down to 1. In the kernel the count is decreased in `sys_unlink()`, while the blocks are freed later in `iput()`, when no name and no reference remain. An xv6 inode has only type, device numbers, link count, size and block addresses, so there are no permissions and no symbolic links.

In the third part, the limits of the file system were measured: names are cut to `DIRSIZ` = 14 characters, the largest file is (12 + 256) × 1024 = 274 432 bytes, and the 2000-block disk has 47 blocks of metadata (boot, superblock, 31 log blocks, 13 inode blocks and 1 bitmap block) and 1953 data blocks.

In the fourth part, the program `find` walked the directory tree recursively and found all files with a given name; with the `-n` option it showed that `./r2` and `cat` are the same inode. The same ideas exist in ext4 on Linux, studied in Lab 5, but xv6 shows them in a few hundred lines of code.

REFERENCES

1. Cox R., Kaashoek F., Morris R. xv6: a simple, Unix-like teaching operating system, chapter 8 "File system". MIT, 2024. [online]. Available: https://pdos.csail.mit.edu/6.1810/2024/xv6/book-riscv-rev4.pdf
2. mit-pdos/xv6-riscv – source code. [online]. Available: https://github.com/mit-pdos/xv6-riscv
3. MIT 6.1810 Operating System Engineering. Lab: Xv6 and Unix utilities (find). [online]. Available: https://pdos.csail.mit.edu/6.1810/2024/labs/util.html
4. Operating Systems course, FCIM/FAF, UTM, 2026–2027. Lab 6: Inside the xv6 File System.